

점프 투 자바

박응용 지음(위키독스 운영자)

이지스퍼블리싱

07

자바의 입출력



07-1 패키지

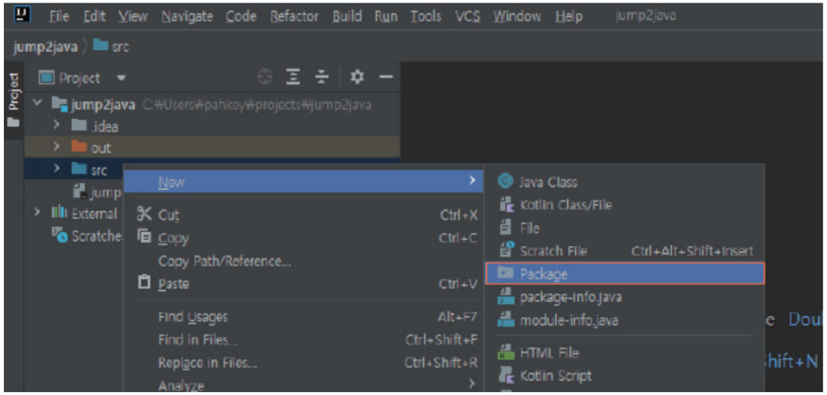
07-2 접근 제어자

07-3 스태틱

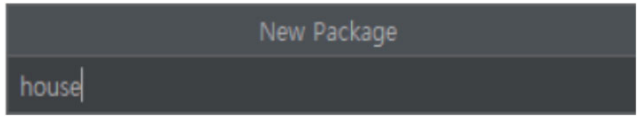
07-1 패키지

▪ 패키지 만들기

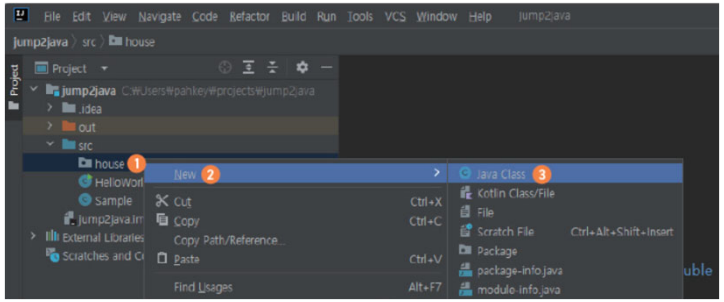
- **패키지**(package)는 비슷한 성격의 클래스들을 모아 놓은 자바의 디렉터리
- HousePark, HouseKim 등의 클래스를 house라는 패키지로 분류하기



src 디렉터리 위에서 마우스 오른쪽 버튼 → [New → Package] 선택



패키지명에 house 입력



house 패키지 위에서 마우스 오른쪽 버튼 → [New → Java Class] 선택

07-1 패키지

▪ 패키지 만들기

- house 패키지에서 클래스를 생성하면 다음처럼 package house;와 같은 문장이 자동으로 삽입됨

Do it! 코딩해 보세요 house/HouseKim.java

```
package house;

public class HouseKim {
}
```

Do it! 코딩해 보세요 house/HousePark.java

```
package house;

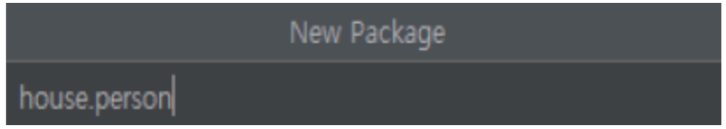
public class HousePark {
}
```

- package는 이 파일이 어떤 패키지의 파일인지 알려 주는 역할

07-1 패키지

▪ 서브 패키지란?

- 기본 패키지 안에 존재하는 하위 패키지
- 기본 패키지 내의 클래스들을 분류하여 체계적으로 관리하고, 가독성을 향상시킬 수 있음
- house 패키지 위에서 마우스 오른쪽 버튼
  → [New → Package] 선택
  → house.person 서브 패키지 생성



- person 패키지 위에서 마우스 오른쪽 버튼
  → [New → Java Class] 선택
  → EungYongPark 클래스 생성

Do it! 코딩해 보세요 — house/person/EungYongPark.java

```
package house.person;

public class EungYongPark {
}
```

07-1 패키지

패키지 사용기

- 다른 클래스에서 HouseKim 클래스 사용하기

1. import 하기

Do it! 코딩해 보세요 Sample.java

```
import house.HouseKim;

public class Sample {
    public static void main(String[] args) {
        HouseKim kim = new HouseKim();
    }
}
```

07-1 패키지

패키지 사용기

- 다른 클래스에서 HouseKim 클래스 사용하기

2. * 기호 이용

Do it! 코딩해 보세요

```
import house.*;

public class Sample {
    public static void main(String[] args) {
        HouseKim kim = new HouseKim();
        HousePark park = new HousePark();
    }
}
```

07-1 패키지

▪ 패키지 사용기

▪ 다른 클래스에서 HouseKim 클래스 사용하기

3. HouseKim과 동일한 패키지 내에 있는 클래스 경우

Do it! 코딩해 보세요

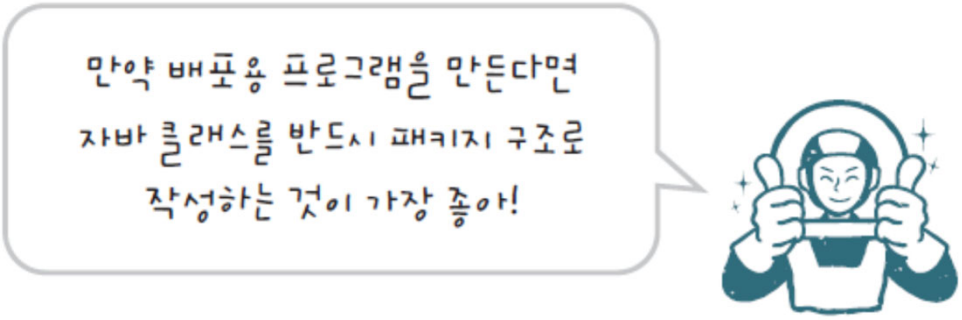
```
package house;

public class HousePark {
    public static void main(String[] args) {
        HouseKim kim = new HouseKim();  // HouseKim 사용을 위해서 import가 필요없다.
    }
}
```

07-1 패키지

패키지를 사용하는 이유

- 클래스의 분류가 용이하다(비슷한 것끼리 묶는다).
- 패키지가 다르다면 동일한 클래스명을 사용할 수 있다.



07-2 접근 제어자

- **접근 제어자(access modifier)**
  - 변수나 메서드의 사용 권한을 설정
  - 접근 제어자의 종류
    - private
    - default
    - protected
    - public
  - private < default < protected < public순으로 보다 많은 접근을 허용

07-2 접근 제어자

- **private**
  - private이 붙은 변수나 메서드는 해당 클래스 안에서만 접근 가능

Do it! 코딩해 보세요 Sample.java

```
public class Sample {
    private String secret;
    private String getSecret() {
        return this.secret;
    }
}
```

07-2 접근 제어자

default

- 접근 제어자를 별도로 설정하지 않으면 default 접근 제어자가 자동으로 설정됨
- default 접근 제어자가 설정되면 동일한 패키지 안에서만 접근 가능

Do it! 코딩해 보세요 house/HouseKim.java

```
package house;  // 패키지가 동일하다.
public class HouseKim {
    String lastname = "kim";  // lastname은 default 접근 제어자로 설정했다.
}
```

07-2 접근 제어자

default

- 접근 제어자를 별도로 설정하지 않으면 default 접근 제어자가 자동으로 설정됨
- default 접근 제어자가 설정되면 동일한 패키지 안에서만 접근 가능

Do it! 코딩해 보세요 house/HousePark.java

```
package house;  // 패키지가 동일하다.

public class HousePark {
    String lastname = "park";

    public static void main(String[] args) {
        HouseKim kim = new HouseKim();
        System.out.println(kim.lastname);  // HouseKim 클래스의 lastname 변수를 사용할 수 있다.
    }
}
```

실행 결과

```
kim
```

07-2 접근 제어자

protected

- protected가 붙은 변수나 메서드는 동일 패키지의 클래스 또는 해당 클래스를 상속받은 클래스에서만 접근 가능

Do it! 코딩해 보세요 house/HousePark.java

```
package house;  // 패키지가 서로 다르다.

public class HousePark {
    protected String lastname = "park";
}
```

07-2 접근 제어자

protected

- protected가 붙은 변수나 메서드는 동일 패키지의 클래스 또는 해당 클래스를 상속받은 클래스에서만 접근 가능

Do it! 코딩해 보세요 house/person/EungYongPark.java

```
package house.person;  // 패키지가 서로 다르다.

import house.HousePark;

public class EungYongPark extends HousePark {  // HousePark을 상속했다.
    public static void main(String[] args) {
        EungYongPark eyp = new EungYongPark();
        System.out.println(eyp.lastname);  // 상속한 클래스의 protected 변수는 접근이 가능
하다.
    }
}
```

실행 결과

```
park
```

07-2 접근 제어자

public

- public 접근 제어자가 붙은 변수나 메서드는 어떤 클래스에서도 접근 가능

Do it! 코딩해 보세요 house/HousePark.java

```
package house;

public class HousePark {
    protected String lastname = "park";
    public String info = "this is public message.";
}
```

07-2 접근 제어자

public

- public 접근 제어자가 붙은 변수나 메서드는 어떤 클래스에서도 접근 가능

Do it! 코딩해 보세요 Sample.java

```
import house.HousePark;

public class Sample {
    public static void main(String[] args) {
        HousePark housePark = new HousePark();
        System.out.println(housePark.info);
    }
}
```

실행 결과

```
this is public message.
```

07-3 스태틱

static 변수

- 클래스에서 공유되는 변수나 메서드를 정의할 때 사용
- lastname 변수에 static 키워드를 붙이면 자바는 메모리 할당을 딱 한 번만 하게 되어 메모리를 적게 사용할 수 있음

Do it! 코딩해 보세요

```
class HouseLee {
    static String lastname = "이";
}

public class Sample {
    public static void main(String[] args) {
        HouseLee lee1 = new HouseLee();
        HouseLee lee2 = new HouseLee();
    }
}
```

07-3 스태틱

▪ static 변수

- static으로 설정하면 같은 메모리 주소만을 바라보기 때문에 static 변수 값을 공유 가능

Do it! 코딩해 보세요

```
class Counter  {
    int count = 0;
    Counter() {
        this.count++;
        System.out.println(this.count);
    }
}

public class Sample {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
    }
}
```

실행 결과

```
1
1
```

07-3 스태틱

static 변수

- 변수에 쓰는 static 키워드는 메모리의 효율을 높이기 위한 목적보다는 공유의 목적으로 훨씬 더 많이 사용

Do it! 코딩해 보세요

```
class Counter {
    static int count = 0;
    Counter() {
        count++;  // count는 더 이상 객체 변수가 아니므로 this를 제거하는 것이 좋다.
        System.out.println(count);  // this 제거
    }
}

public class Sample {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
    }
}
```

실행 결과

```
1
2
```

07-3 스태틱

static 메서드

- static이라는 키워드가 메서드 앞에 붙으면 스태틱 메서드(static method)

Do it! 코딩해 보세요

```
class Counter  {
    static int count = 0;
    Counter() {
        count++;
        System.out.println(count);
    }

    public static int getCount() {
        return count;
    }
}
```

```
public class Sample {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();

        System.out.println(Counter.getCount());  // 스태틱 메서드는 클래스를 이용하여 호출
    }
}
```

실행 결과

```
1
2
2
```

07-3 스태틱

static 메서드

- '오늘의 날짜 구하기', '숫자에 콤마 추가하기' 등 유틸리티성 메서드를 작성할 때 많이 사용

Do it! 코딩해 보세요

```
import java.text.SimpleDateFormat;
import java.util.Date;

class Util {
    public static String getCurrentDate(String fmt) {
        SimpleDateFormat sdf = new SimpleDateFormat(fmt);
        return sdf.format(new Date());
    }
}

public class Sample {
    public static void main(String[] args) {
        System.out.println(Util.getCurrentDate("yyyyMMdd"));  // 오늘 날짜 출력
    }
}
```

07-3 스태틱

▪ 싱글톤 패턴

- 싱글톤(singleton)은 단 하나의 객체만을 생성하게 강제하는 디자인 패턴
- 클래스를 통해 생성할 수 있는 객체가 한 개만 되도록 만드는 것

Do it! 코딩해 보세요

```
class Singleton {
    private static Singleton one;
    private Singleton() {
    }

    public static Singleton getInstance() {
        if(one == null) {
            one = new Singleton();
        }
        return one;
    }
}
```

```
public class Sample {
    public static void main(String[] args) {
        Singleton singleton1 = Singleton.getInstance();
        Singleton singleton2 = Singleton.getInstance();
        System.out.println(singleton1 == singleton2);  // true 출력
    }
}
```

실행 결과

```
true
```



감사합니다.